

# A Hierarchical Time-Aware Transformer for Flow-Level Network Intrusion Detection

Xiaoyan Xiong

Student ID: S3693414

University of Twente

April 2026

## 1 Introduction

Modern networked systems, including cloud platforms, enterprise infrastructures, mobile networks, and Internet-of-Things (IoT) deployments, rely on continuous traffic monitoring to ensure security and availability. In this context, Network Intrusion Detection Systems (NIDS) are essential for identifying malicious activities such as distributed denial-of-service (DDoS), scanning, exploitation attempts, and multi-stage intrusions [1, 2]. This task is increasingly challenging due to the rapid growth of network traffic, the rising sophistication and distribution of attacks, and the widespread adoption of encryption, which limits payload inspection and shifts detection toward metadata- and behavior-driven analysis [3, 4].

Signature-based NIDS, such as Snort and Suricata, remain widely used due to their interpretability and effectiveness against known threats [1, 5]. However, their dependence on manually curated rules limits their ability to detect zero-day attacks, evolving variants, and context-dependent behaviors, while also raising scalability concerns as traffic rates and rule sets grow [2, 6]. Learning-based approaches address some of these limitations but typically rely on flow-level statistical features, which discard fine-grained sequential information such as packet ordering, direction changes, and inter-packet timing [2, 7]. This creates a trade-off between scalability and representational fidelity.

Transformer-based models have recently been applied to NIDS, demonstrating the ability to capture long-range dependencies [8, 9]. However, most approaches treat each flow as a flat, aggregated representation, introducing a structural mismatch with network traffic, which is inherently hierarchical. A flow consists of temporally ordered packets whose interactions encode critical behavioral patterns, and aggregation can obscure signals relevant to stealthy or temporally structured attacks [7, 9]. Moreover, existing models often rely on coarse or implicit temporal encoding and fail to capture irregular timing dynamics. Many attacks also span multiple flows, requiring both intra-flow packet modeling and inter-flow contextual reasoning [10].

Meanwhile, self-supervised learning and pretrained models offer new opportunities for learning from large-scale, weakly labeled traffic [11, 12, 4]. However, directly applying NLP architectures is insufficient, as packet sequences are protocol-structured, heterogeneous, and strongly

grounded in real time, with semantics shaped by timing, directionality, and protocol interactions.

Motivated by these limitations, this project investigates a packet-sequence-based framework for NIDS that preserves the temporal and hierarchical structure of network traffic. Each bidirectional flow is modeled as an ordered packet sequence, and representations are learned from packet headers and metadata without relying on payload content, making the approach suitable for encrypted environments [3, 4]. The framework is guided by three principles: (1) protocol-aware packet representation, (2) time-aware intra-flow modeling using continuous-time information, and (3) cross-flow contextual modeling to capture dependencies across related flows.

The central research problem is: *how can intrusion detection benefit from packet-sequence modeling while preserving the temporal and hierarchical structure of network traffic?* This study addresses two research questions:

- **RQ1:** Does time-aware packet-sequence modeling produce more informative representations than flow-level aggregation, particularly for attacks characterized by timing, direction, and burst patterns?
- **RQ2:** Does incorporating cross-flow context improve detection of attacks distributed across multiple flows compared to independent per-flow analysis?

This work aims to develop a representation learning framework grounded in the intrinsic structure of network traffic, contributing a more principled NIDS design for encrypted, high-volume, and weakly labeled environments, along with an empirical analysis of modeling granularity, detection performance, and computational efficiency.

## 2 Related Work

Research on network intrusion detection has evolved along several major technical paradigms, including rule-based systems, classical machine learning, deep learning, Transformer-based sequence modeling, and more recent self-supervised and foundation-model approaches. Rather than treating these directions independently, this section organizes them by modeling assumptions and highlights their limitations in capturing the structural and temporal properties of network traffic.

### 2.1 Rule-based and classical learning-based intrusion detection

Rule-based Network Intrusion Detection Systems (NIDS), such as Snort and Suricata, remain widely deployed due to their interpretability, mature ecosystems, and strong performance against known threats [1, 5]. However, their reliance on predefined signatures fundamentally limits their ability to detect zero-day attacks, polymorphic variants, and context-dependent malicious behaviors [2, 13]. Moreover, as traffic volume and rule sets grow, maintaining real-time performance becomes increasingly challenging, particularly in high-speed and encrypted environments [6, 14].

Classical machine learning approaches were introduced to improve adaptability by learning from data rather than fixed rules. Methods such as decision trees, random forests, support vector machines, and gradient boosting operate on handcrafted flow-level features [15, 16, 17]. While these models are efficient and often perform well on benchmark datasets, they depend heavily on feature engineering and aggregation. As a result, they discard packet-level ordering, timing irregularities, and interaction patterns, which are essential for detecting stealthy or temporally structured attacks [15, 6].

## 2.2 Deep learning for representation learning and sequence modeling

Deep learning methods aim to reduce reliance on manual feature engineering and improve representation learning from traffic data. Convolutional neural networks (CNNs), multilayer perceptrons (MLPs), recurrent neural networks (RNNs), and autoencoder-based architectures have been widely explored [18, 19, 20, 21, 22]. Recurrent models such as LSTM-based detectors explicitly capture sequential dependencies and demonstrate improved performance over purely statistical approaches [18, 21]. Autoencoder-based systems further enable lightweight anomaly detection under resource constraints [20].

However, most deep learning approaches still operate on aggregated flow-level inputs or fixed-size feature representations. CNN- and MLP-based models often ignore temporal ordering, while RNN-based approaches suffer from limited scalability and degraded efficiency on long sequences [19, 22]. Consequently, these methods only partially capture the fine-grained packet interactions and temporal dynamics that characterize modern attack behaviors.

## 2.3 Transformer-based NIDS and time-aware traffic modeling

Transformer architectures have recently become a dominant paradigm for sequence modeling due to their ability to capture long-range dependencies with parallel computation [8]. Several studies have adapted Transformers to NIDS, demonstrating improved performance over earlier deep learning approaches [23, 7, 9, 24]. For example, RTIDS applies Transformer encoders to tabular traffic features, while GTID incorporates temporal information through n-gram representations and time-aware attention mechanisms [23, 7]. FlowTransformer further provides a modular framework for evaluating encoding strategies in flow-based intrusion detection [9].

Despite these advances, most Transformer-based NIDS treat each flow as a flattened or aggregated representation. This introduces a structural mismatch with network traffic, which is inherently hierarchical and packet-driven. A flow consists of temporally ordered packets with protocol-specific semantics, directionality, and inter-packet timing patterns. Aggregation at the flow level can obscure these signals, particularly for stealthy or bursty attacks [7, 9]. In addition, standard positional encoding fails to capture continuous-time dynamics, limiting the model’s ability to represent irregular timing patterns that are often critical in security scenarios.

Beyond architectural design, another important challenge in NIDS is the dependence on high-quality labeled data. Since attack labels are often sparse, noisy, or biased, recent studies have increasingly explored self-supervised learning and pretraining as ways to improve representation learning and generalization.

## 2.4 Self-supervised learning, contrastive learning, and pretraining

Self-supervised learning (SSL) has emerged as a promising direction for NIDS, especially in scenarios where labeled data is scarce or unreliable. Recent work explores masked modeling, contrastive learning, and pretraining objectives to learn generalizable traffic representations [11, 25, 26, 27, 12]. For instance, masked autoencoding methods reconstruct missing traffic features, while contrastive approaches learn invariant representations across augmented views of network flows [25, 26]. Graph-based SSL further incorporates relational structure into representation learning [11].

While these approaches improve label efficiency and robustness, most operate on flow-level or graph-level abstractions. They rarely treat packet sequences as first-class modeling units and often overlook continuous-time dynamics, protocol structure, and hierarchical dependencies. As a result, SSL improves representation quality but does not fully address the structural mismatch between models and network traffic.

## 2.5 Encrypted traffic analysis and payload-limited settings

The widespread adoption of encryption has fundamentally changed network traffic analysis, as payload inspection is often unavailable [3, 28, 29]. Consequently, modern NIDS must rely on packet headers, metadata, and traffic patterns rather than deep packet inspection. This has led to increasing interest in encrypted traffic classification using deep learning and pretrained models [30, 29].

ET-BERT demonstrates that large-scale pretraining on encrypted traffic can significantly improve downstream performance [30]. However, most studies in this domain focus on application classification rather than adversarial intrusion detection. Furthermore, they typically operate at the flow or session level and do not explicitly model packet-level temporal structure or cross-flow dependencies, limiting their applicability to more complex intrusion scenarios.

## 2.6 Graph-based, cross-flow, and LLM-inspired traffic modeling

Graph-based approaches model network traffic as relational structures, capturing dependencies among flows, hosts, or communication events [31, 32, 11]. These methods are particularly effective for detecting coordinated or distributed attacks, as they explicitly encode interaction patterns across entities. However, they often abstract away intra-flow packet dynamics, focusing instead on higher-level relationships.

More recently, large language model (LLM)-inspired approaches have been explored for traffic analysis. DoLLM leverages pretrained language models to capture contextual relationships across flow sequences and demonstrates effectiveness for distributed attack detection [10]. However, such approaches remain in early stages and face fundamental challenges due to differences between network traffic and natural language, including protocol structure, temporal grounding, and heterogeneous feature semantics [33]. Simply transferring NLP architectures does not fully address these domain-specific requirements.

## 2.7 Summary of limitations and positioning of this work

Across these categories, prior work exhibits three fundamental limitations. First, flow-level representations obscure packet-level temporal dynamics and interaction patterns. Second, existing sequence models do not fully capture hierarchical structure and continuous-time behavior. Third, contextual approaches model inter-flow relationships but often neglect intra-flow packet interactions.

Therefore, the key gap is not the absence of more powerful models, but the lack of a unified framework that simultaneously captures: (1) packet-level structure, (2) real-time temporal dynamics, and (3) cross-flow contextual dependencies. This gap directly motivates the proposed packet-sequence-based, time-aware, and context-aware NIDS framework.

## 3 Problem Formulation

Network Intrusion Detection Systems (NIDS) aim to infer whether observed network traffic is benign or malicious from sequential evidence. Unlike conventional approaches that directly map handcrafted flow-level features to labels, we formulate intrusion detection as a *hierarchical sequence modeling problem* over packet-level observations and flow-level contexts. This formulation is motivated by recent advances in sequence modeling, which highlight the importance of capturing both temporal dependencies and contextual relationships in traffic data [8, 9, 30].

### 3.1 Input Space and Notation

Let the dataset be defined as

$$\mathcal{D} = \{(\mathcal{S}_i, y_i)\}_{i=1}^N, \quad (1)$$

where  $N$  is the number of flows,  $\mathcal{S}_i$  denotes the input associated with flow  $i$ , and  $y_i \in \{1, \dots, C\}$  is the corresponding class label.

Each flow  $i$  consists of a sequence of  $T_i$  packets:

$$\mathcal{F}_i = \{p_{i,1}, p_{i,2}, \dots, p_{i,T_i}\}. \quad (2)$$

Each packet is represented as:

$$x_{i,t} = [h_{i,t}, s_i, \tau_{i,t}], \quad (3)$$

where  $h_{i,t}$  denotes packet header features,  $s_i$  denotes flow-level statistical features shared across packets, and  $\tau_{i,t}$  encodes temporal information such as timestamps or inter-arrival times.

Thus, the packet sequence for flow  $i$  is:

$$X_i = \{x_{i,1}, x_{i,2}, \dots, x_{i,T_i}\}. \quad (4)$$

To capture cross-flow dependencies, we define a context window:

$$\mathcal{C}_i = \{X_{i-K}, \dots, X_i, \dots, X_{i+K}\}, \quad (5)$$

where  $K$  controls the size of neighboring flows considered.

### 3.2 Intra-flow Sequence Modeling

To encode packet-level dependencies, each packet feature is first projected into a latent space:

$$e_{i,t} = W_e x_{i,t} + b_e. \quad (6)$$

We then incorporate temporal dynamics through a time-aware positional encoding:

$$\tilde{e}_{i,t} = e_{i,t} + \phi_{\text{tp}}(\tau_{i,t}, t), \quad (7)$$

where  $\phi_{\text{tp}}(\cdot)$  encodes both packet order and timing information.

The sequence is processed by a Transformer encoder:

$$H_i^{\text{intra}} = \text{Transformer}_{\text{intra}}(\tilde{e}_{i,1}, \dots, \tilde{e}_{i,T_i}), \quad (8)$$

and aggregated into a flow representation:

$$z_i^{\text{intra}} = \text{Pool}(H_i^{\text{intra}}). \quad (9)$$

### 3.3 Inter-flow Context Modeling

To model dependencies across flows, we construct a sequence of intra-flow embeddings:

$$Z_i = \{z_{i-K}^{\text{intra}}, \dots, z_i^{\text{intra}}, \dots, z_{i+K}^{\text{intra}}\}. \quad (10)$$

A second Transformer encoder produces context-aware representations:

$$H_i^{\text{inter}} = \text{Transformer}_{\text{inter}}(Z_i), \quad (11)$$

and the embedding corresponding to the target flow is denoted as:

$$z_i^{\text{inter}} \in R^d. \quad (12)$$

### 3.4 Detection Function

The final prediction is computed as:

$$\hat{y}_i = f(X_i, \mathcal{C}_i; \theta) = \sigma(g(z_i^{\text{inter}})), \quad (13)$$

where  $g(\cdot)$  is a classification head and  $\sigma(\cdot)$  is the sigmoid or softmax function.

The overall model can be summarized as:

$$f(X_i, \mathcal{C}_i; \theta) = f_{\text{cls}}\left(f_{\text{inter}}\left(f_{\text{intra}}(X_i), \{f_{\text{intra}}(X_j)\}_{j \in \mathcal{N}(i)}\right)\right). \quad (14)$$

### 3.5 Optimization Objective

To address class imbalance, we adopt focal loss:

$$\mathcal{L}_{\text{focal}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C \alpha_c (1 - \hat{p}_{i,c})^\gamma y_{i,c} \log(\hat{p}_{i,c}). \quad (15)$$

The overall optimization problem is:

$$\min_{\theta} \mathcal{L}_{\text{focal}} + \lambda \|\theta\|_2^2. \quad (16)$$

### 3.6 Problem Summary

In summary, we aim to learn a hierarchical detection function that:

- models packet-level sequential dependencies within each flow;
- captures contextual relationships across flows;
- incorporates temporal dynamics into representation learning;
- remains robust under class imbalance.

## 4 Viable Approach

To address the limitations identified in the previous sections, we propose a hierarchical Transformer-based NIDS framework that models both *intra-flow packet dependencies* and *inter-flow contextual relationships*. The key idea is that intrusion patterns emerge not only from packet-level sequential behavior within a flow, but also from dependencies across related flows. Accordingly, the proposed approach decomposes intrusion detection into two coupled stages: intra-flow representation learning and inter-flow context modeling.

### 4.1 Overall Framework

The overall pipeline is illustrated in Fig. 1.

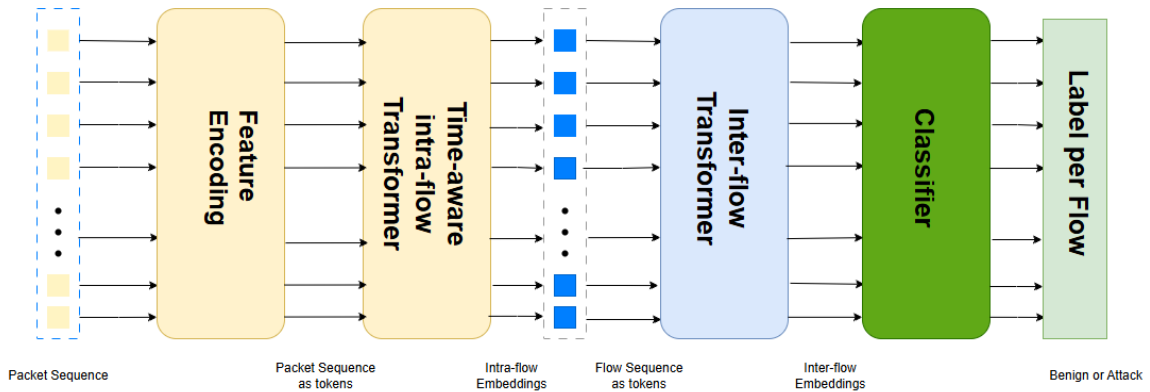


Figure 1: The overall framework

For each target flow, packet observations are transformed into token representations that integrate packet header features, flow statistics, and temporal information. These tokens are processed by a first Transformer encoder to model sequential dependencies within the flow. The resulting embeddings are then passed to a second Transformer encoder to capture dependencies among neighboring flows. Finally, a classification head predicts the attack label using focal loss.

This design leverages the ability of Transformers to model long-range dependencies while extending existing approaches to incorporate both temporal dynamics and cross-flow context.

## 4.2 Stage I: Packet-level Feature Construction

For the  $i$ -th flow  $\mathcal{F}_i = \{p_{i,1}, \dots, p_{i,T_i}\}$ , each packet is represented as:

$$x_{i,t} = [h_{i,t}; s_i; \tau_{i,t}], \quad (17)$$

where  $h_{i,t}$  denotes packet header features,  $s_i$  denotes flow-level statistics, and  $\tau_{i,t}$  encodes temporal information.

A linear embedding layer maps each packet into the model space:

$$e_{i,t} = W_x x_{i,t} + b_x. \quad (18)$$

This representation integrates fine-grained packet information with global flow context, enabling richer traffic characterization.

## 4.3 Stage II: Time-aware Intra-flow Transformer

To model packet interactions within a flow, we employ a time-aware Transformer encoder:

$$\tilde{e}_{i,t} = e_{i,t} + \phi_{\text{time}}(\tau_{i,t}, t), \quad (19)$$

where  $\phi_{\text{time}}(\cdot)$  encodes both packet order and temporal spacing.

The encoded sequence is processed as:

$$H_i^{\text{intra}} = \text{Transformer}_{\text{intra}}(\tilde{e}_{i,1}, \dots, \tilde{e}_{i,T_i}), \quad (20)$$

followed by:

$$z_i^{\text{intra}} = \text{Pool}(H_i^{\text{intra}}). \quad (21)$$

## 4.4 Stage III: Inter-flow Context Modeling

To capture dependencies across flows, we construct a contextual sequence:

$$Z_i = \{z_{i-K}^{\text{intra}}, \dots, z_i^{\text{intra}}, \dots, z_{i+K}^{\text{intra}}\}. \quad (22)$$

This sequence is processed by a second Transformer:

$$H_i^{\text{inter}} = \text{Transformer}_{\text{inter}}(Z_i), \quad (23)$$



and the target representation is denoted as  $z_i^{\text{inter}}$ .

This stage enables context-aware detection by modeling relationships among neighboring flows.

## 4.5 Stage IV: Classification and Optimization

The final prediction is:

$$\hat{y}_i = \text{Softmax}(W_c z_i^{\text{inter}} + b_c). \quad (24)$$

To address class imbalance, focal loss is adopted:

$$\mathcal{L}_{\text{focal}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C \alpha_c (1 - \hat{p}_{i,c})^\gamma y_{i,c} \log(\hat{p}_{i,c}). \quad (25)$$

The overall objective is:

$$\min_{\theta} \mathcal{L}_{\text{focal}} + \lambda \|\theta\|_2^2. \quad (26)$$

Having defined the model architecture and optimization objective, the next issue is how to construct packet-level and flow-level inputs suitable for training and evaluation.

## 4.6 Data Collection and Preprocessing

The evaluation will be conducted using both public and internal datasets to balance reproducibility and realism. Specifically, the public benchmark **CIC-IDS2017** will be used for standardized comparison, while internal PCAP datasets (e.g., `firewall_test` and `bacore001`) will support validation under real traffic conditions.

Raw PCAP files will be processed using a flow reconstruction pipeline (e.g., Suricata) to extract structured representations. The preprocessing pipeline includes:

1. flow reconstruction from packet streams;
2. packet header feature extraction;
3. flow-level statistic computation;
4. temporal feature derivation (e.g., inter-arrival time);
5. sequence construction and length normalization;
6. feature normalization;
7. label cleaning and mapping.

**Sequence Length Handling.** Network traffic sequences exhibit a heavy-tailed length distribution, where the majority of flows are short while a small number are extremely long. Directly modeling full-length sequences would lead to excessive computational cost and memory usage.

To address this, a maximum sequence length  $L$  will be introduced for both packet-level and flow-level sequences. For sequences with length  $\leq L$ , all elements are retained. For sequences exceeding  $L$ , truncation or subsampling strategies are applied.

Following common practice in prior work, this study adopts *head truncation* (i.e., retaining the first  $L$  elements) as the primary strategy, based on the observation that attack-related patterns often emerge in early stages of a connection.

To further evaluate the impact of long-sequence handling, alternative strategies will also be explored:

- **Head-only:** retain the first  $L$  elements;
- **Head-tail:** retain the first and last segments of the sequence;
- **Sampling-based:** randomly sample  $L$  elements from the full sequence.

Shorter sequences will be padded to length  $L$  using masking to ensure compatibility with Transformer-based architectures.

Where necessary, additional internal datasets with larger scale and broader attack coverage will be incorporated to improve robustness and generalization.

**Label Quality and Bias Mitigation.** Labels generated by Suricata may introduce bias, as they are derived from predefined detection rules. This creates a risk that the model learns to replicate rule-based patterns rather than generalizing to underlying attack behaviors.

To mitigate this issue, several strategies are adopted:

- **Raw feature learning:** The model is trained on packet-level and temporal features rather than rule-derived indicators, reducing direct leakage of detection logic.
- **Feature isolation:** Any metadata or fields directly produced by Suricata (e.g., rule IDs, alert signatures) are excluded from model inputs.
- **Diverse data sources:** In addition to Suricata-labeled internal data, publicly labeled datasets (e.g., CIC-IDS2017) are incorporated to reduce dependence on a single rule-based labeling scheme.

## 4.7 Training and Evaluation

The dataset will be split into training, validation, and test sets (e.g., 70/10/20), with class stratification where possible. All preprocessing statistics will be computed on the training set only to avoid data leakage.

Model hyperparameters, including sequence length  $L$ , embedding size, Transformer depth, attention heads, context window  $K$ , and focal loss parameters, will be tuned using validation data. Early stopping will be applied based on validation performance.

Evaluation will include both binary and multiclass classification settings. In addition to Accuracy, Precision, Recall, and F1-score, the following metrics will be reported:

$$\text{FPR} = \frac{FP}{FP + TN}, \quad \text{TPR} = \frac{TP}{TP + FN}, \quad \text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c. \quad (27)$$

To validate the contribution of each component, ablation studies will compare: (i) full model vs. flow-only baseline, (ii) with/without time-aware encoding, (iii) with/without inter-flow

modeling, (iv) focal loss vs. cross-entropy, (v) different sequence handling strategies (head-only, head-tail, sampling).

**Bias-Aware Evaluation.** Since labels are partially generated using Suricata, there is a risk that the model learns to approximate rule-based detection behavior rather than generalizable attack patterns. To assess and mitigate this issue, additional evaluation strategies are introduced:

- **Cross-dataset evaluation:** Models trained on Suricata-labeled internal data will be evaluated on publicly labeled datasets (e.g., CIC-IDS2017) to assess generalization across different labeling schemes.
- **Unseen attack evaluation:** Where possible, testing will include attack scenarios or traffic patterns not explicitly covered by Suricata rules.

## 4.8 Summary

The proposed approach is a hierarchical and context-aware NIDS framework that integrates packet-level modeling, temporal encoding, and inter-flow reasoning. By combining these components within a unified Transformer-based architecture, the method directly addresses limitations in sequential modeling, temporal awareness, contextual reasoning, and class imbalance.

## 5 Initial Results of Baselines

At the proposal stage, this section aims to demonstrate that a concrete and feasible evaluation plan has been established, rather than to report finalized empirical results. The target task is **flow-level intrusion classification**, and the baseline design follows the hierarchical structure of the proposed model.

Specifically, the evaluation is organized into three parts: (i) overall model comparison at the flow level, (ii) validation of the intra-flow encoder (Stage I), and (iii) validation of the inter-flow encoder (Stage II).

### 5.1 Baseline Design

Since the final task is flow-level classification, the primary baselines are constructed as flow-level classifiers using statistical flow features. In addition, component-level validation is conducted to isolate the contributions of the two stages in the proposed architecture.

### 5.2 Part I: Overall Flow-level Baselines

The first group of baselines evaluates end-to-end performance at the flow-classification level. All models take flow-level statistical features  $s_i$  as input.

#### 5.2.1 LSTM Flow-level Classifier

$$h_i^{\text{lstm}} = \text{LSTM}(s_i), \quad \hat{y}_i = \text{Softmax}(Wh_i^{\text{lstm}} + b). \quad (28)$$

### 5.2.2 CNN+LSTM Flow-level Classifier

$$u_i = f_{\text{cnn}}(s_i), \quad h_i^{\text{cnn+lstm}} = f_{\text{lstm}}(u_i), \quad \hat{y}_i = \text{Softmax}(Wh_i^{\text{cnn+lstm}} + b). \quad (29)$$

### 5.2.3 Transformer Flow-level Classifier

$$H_i^{\text{trans}} = \text{Transformer}(s_i), \quad z_i^{\text{trans}} = \text{Pool}(H_i^{\text{trans}}), \quad \hat{y}_i = \text{Softmax}(Wz_i^{\text{trans}} + b). \quad (30)$$

These baselines provide strong reference points for evaluating the full proposed architecture.

## 5.3 Part II: Stage-I Validation (Intra-flow Encoder)

This part evaluates whether packet-level temporal modeling yields more informative flow representations than direct statistical features. In addition, ablation studies are conducted to isolate the contribution of different temporal encoding strategies.

### 5.3.1 Reference: MLP on Flow Features

$$z_i^{\text{mlp}} = f_{\text{mlp}}(s_i), \quad \hat{y}_i = \text{Softmax}(Wz_i^{\text{mlp}} + b). \quad (31)$$

### 5.3.2 Packet Transformer without Temporal Encoding

Each flow is represented as a packet sequence:

$$X_i = \{x_{i,1}, \dots, x_{i,T_i}\}, \quad (32)$$

and encoded as:

$$h_i^{\text{pkt}} = \text{Transformer}(X_i), \quad \hat{y}_i = \text{Softmax}(Wh_i^{\text{pkt}} + b). \quad (33)$$

This baseline evaluates the effectiveness of packet-level modeling without explicit temporal encoding.

### 5.3.3 Packet Transformer with Positional Encoding Only

$$h_i^{\text{pos}} = \text{Transformer}(X_i + \text{PE}(t)), \quad \hat{y}_i = \text{Softmax}(Wh_i^{\text{pos}} + b). \quad (34)$$

This variant evaluates the contribution of packet order information via sinusoidal positional encoding.

### 5.3.4 Packet Transformer with Log Inter-arrival Time Only

$$h_i^{\text{iat}} = \text{Transformer}(X_i + \phi(\Delta t_{i,t})), \quad \hat{y}_i = \text{Softmax}(Wh_i^{\text{iat}} + b). \quad (35)$$

where  $\Delta t_{i,t}$  denotes the inter-arrival time between consecutive packets, and  $\phi(\cdot)$  is a transformation such as  $\log(1 + \Delta t_{i,t})$ .

This variant evaluates the contribution of continuous-time dynamics without positional encoding.

### 5.3.5 Packet Transformer with Combined Temporal Encoding

$$h_i^{\text{time}} = \text{Transformer}(X_i + \text{PE}(t) + \phi(\Delta t_{i,t})), \quad \hat{y}_i = \text{Softmax}(Wh_i^{\text{time}} + b). \quad (36)$$

This represents the full time-aware intra-flow encoder used in the proposed model.

This comparison is conducted without inter-flow modeling, isolating the effect of intra-flow representation learning and temporal encoding.

These ablations ensure that improvements can be attributed to packet-level modeling, temporal ordering, and continuous-time dynamics, rather than model capacity alone.

## 5.4 Part III: Stage-II Validation (Inter-flow Encoder)

This part evaluates the effectiveness of the second-stage flow-context encoder.

### 5.4.1 Model A: Transformer-based Second Stage

$$h_i^{\text{time}} = \text{TimeAwareTransformer}(X_i), \quad g_i^{\text{trans2}} = \text{Transformer}_2(h_i^{\text{time}}), \quad \hat{y}_i = \text{Softmax}(Wg_i^{\text{trans2}} + b). \quad (37)$$

### 5.4.2 Model B: LSTM-based Second Stage

$$h_i^{\text{time}} = \text{TimeAwareTransformer}(X_i), \quad g_i^{\text{lstm2}} = \text{LSTM}_2(h_i^{\text{time}}), \quad \hat{y}_i = \text{Softmax}(Wg_i^{\text{lstm2}} + b). \quad (38)$$

Since the first-stage encoder is identical, this comparison isolates the impact of the second-stage modeling choice.

## 5.5 Experimental Configuration

To ensure fair comparison:

- all overall baselines use flow-level statistical features;
- the Stage-I reference model uses the same inputs;
- the Stage-I validation model uses packet-level sequences without cross-flow context;
- the Stage-II models share identical first-stage embeddings.

All models will be trained using the same preprocessing pipeline, data splits, and evaluation protocol. Hyperparameters will be tuned on the validation set.

## 5.6 Summary

Although no finalized results are reported, the baseline design demonstrates a clear and testable evaluation strategy. It enables both end-to-end comparison with established flow-level models and component-level analysis aligned with the hierarchical structure of the proposed method.

## 6 Risks and Limitations

Despite the potential advantages of the proposed hierarchical, time-aware Transformer framework, several risks and limitations should be acknowledged.

**Limited Benefit of Packet-Level Modeling** While packet-level sequence modeling provides finer-grained representations compared to flow-level aggregation, it may not always yield significant performance improvements. Many existing datasets and attack patterns can already be effectively captured using flow-level statistical features, and strong baselines such as Random Forest or gradient boosting models may perform competitively. As a result, the added complexity of packet-level modeling may not translate into consistent gains across all attack types. To address this, the evaluation will include both overall and per-attack performance analysis, with particular attention to timing-sensitive and multi-flow attacks where packet-level information is expected to be more beneficial.

**Computational and Deployment Cost** The proposed architecture introduces increased computational overhead due to hierarchical modeling. Intra-flow Transformer encoding scales with packet sequence length, while inter-flow context modeling introduces additional cost proportional to the number of related flows. This may limit scalability and make real-time deployment challenging in high-throughput environments. To mitigate this issue, sequence length and context size will be constrained, and resource usage, including training time, memory consumption, and inference latency, will be reported. Lightweight alternatives, such as LSTM-based context modeling, will also be considered to evaluate performance-efficiency trade-offs.

**Label Bias from Suricata-Based Annotation** The use of Suricata-generated labels introduces a risk that the model may learn to replicate rule-based detection patterns rather than generalizable attack behavior. Even when explicit rule identifiers are excluded, packet-level features may still implicitly encode rule logic. This can lead to overfitting to the labeling mechanism and reduced robustness to unseen attacks. To mitigate this, rule-derived metadata will be excluded from model inputs, and evaluation will include externally labeled datasets and cross-dataset testing to assess generalization.

**Dataset Distribution Shift** The combination of public datasets (e.g., CIC-IDS2017) and internally collected PCAP data introduces potential distribution mismatches in terms of traffic characteristics, attack types, and labeling procedures. Models trained on one dataset may not generalize well to another due to domain shift. To address this limitation, the study will clearly separate in-domain and cross-domain evaluations, report dataset-specific performance, and, where feasible, conduct cross-dataset experiments to assess robustness under varying traffic conditions.

**Limited Interpretability** Compared to signature-based systems, which provide explicit and interpretable detection rules, the proposed deep learning model offers limited transparency. Although attention mechanisms may provide some insights into influential packets or flows, they do not constitute fully reliable explanations of model decisions. This limitation may reduce

practical applicability in security operations where interpretability is important. To partially address this, qualitative analysis and case studies will be conducted to illustrate model behavior on selected examples, while acknowledging that interpretability remains an open challenge.

**Summary** In summary, the proposed approach involves trade-offs between representational power, computational cost, generalization ability, and interpretability. These limitations will be explicitly considered in both experimental design and result interpretation to ensure a balanced and rigorous evaluation.

## 7 Planning

This project will be carried out in an iterative manner rather than as a strictly linear process. Several stages, particularly data preprocessing, model design, and experimental evaluation, may require rework and repeated refinement as new findings emerge. In addition, thesis writing will proceed in parallel with the technical work throughout the project, so that methodological decisions, experimental observations, and revised designs can be documented continuously rather than postponed until the end.

### 7.1 Week-by-week Work Plan

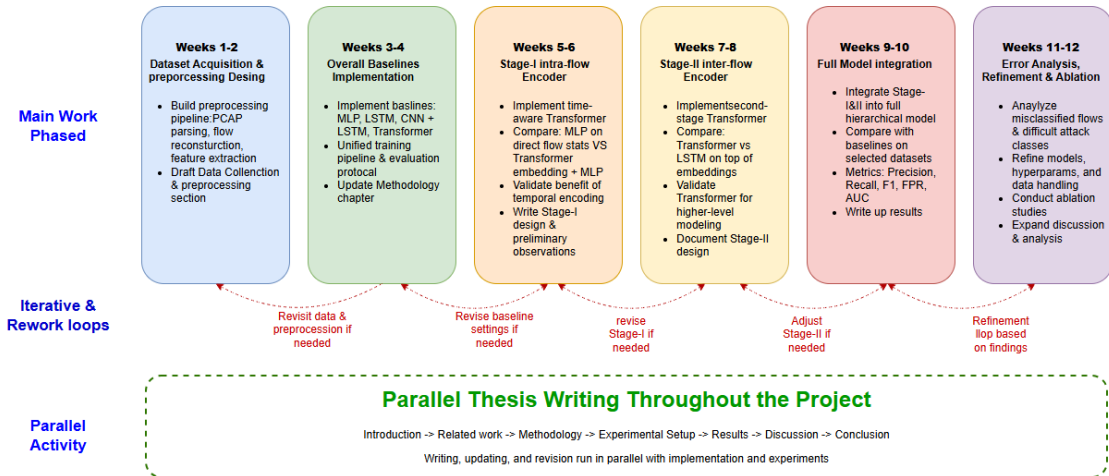


Figure 2: Work Plan

### 7.2 Work Plan Characteristics

It should be noted that this timeline is intentionally iterative. Model development and experimental findings may require repeated adjustment of feature engineering, module design, and evaluation settings. Therefore, some activities may overlap or loop back to earlier phases. In particular, preprocessing, Stage-I design, and Stage-II design are expected to be the most iterative parts of the project. Meanwhile, thesis writing will continue throughout the process in parallel with implementation and experimentation, which will help ensure that the final dissertation is developed continuously rather than written only after all experiments are completed.

## References

- [1] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA)*, 1999.
- [2] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*, pages 305–316, 2010.
- [3] A. Sharma et al. A survey on encrypted network traffic: A comprehensive review. *Computer Networks*, page 110984, 2025.
- [4] Wenqi Dong, Jing Yu, Xinjie Lin, Gaopeng Gou, and Gang Xiong. Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review. *Neurocomputing*, page 128444, 2025.
- [5] Open Information Security Foundation. Suricata user guide. <https://docs.suricata.io/>, 2026.
- [6] Ali Hozouri, Abbas Mirzaei, and Mehdi Effatparvar. A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. *Discover Applied Sciences*, 2025.
- [7] Xueying Han, Susu Cui, Song Liu, Zhigang Lu, and Bo Jiang. Network intrusion detection based on n-gram frequency and time-aware transformer. *Computers & Security*, 127:103171, 2023.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [9] Liam Daly Manocchio, Siamak Layeghy, Wai Weng Lo, Gayan K. Kulatilleke, Mohanad Sarhan, and Marius Portmann. Flowtransformer: A transformer framework for flow-based network intrusion detection systems. *Expert Systems with Applications*, 2024.
- [10] Qingyang Li, Yihang Zhang, Zhidong Jia, Yannan Hu, Lei Zhang, Jianrong Zhang, Yongming Xu, Yong Cui, Zongming Guo, and Xinggong Zhang. Dollm: How large language models understanding network flow data to detect carpet bombing ddos. *arXiv preprint arXiv:2405.07638*, 2024.
- [11] Renjie Xu, Guangwei Wu, Weiping Wang, Xing Gao, An He, and Zhengpeng Zhang. Applying self-supervised learning to network intrusion detection for network flows with graph neural network. *arXiv preprint arXiv:2403.01501*, 2024.
- [12] Ke Xu, Xixi Zhang, Yu Wang, Tomoaki Ohtsuki, Bamidele Adebisi, Hikmet Sari, and Guan Gui. Self-supervised learning malware traffic classification based on masked autoencoder. *IEEE Internet of Things Journal*, 11(10):17330–17340, 2024.



- [13] Mohammad Ahmed, Abdun Naser Mahmood, and Jiankun Hu. A survey of network anomaly detection techniques. *Journal of Network and Computer Applications*, 60:19–31, 2016.
- [14] M. Grossi, F. Alfonsi, M. Prandini, and A. Gabrielli. A high throughput intrusion detection system (ids) to enhance the security of data transmission among research centers. *Journal of Instrumentation*, 18(12):C12017, dec 2023.
- [15] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016.
- [16] Shadi Aljawarneh, Muneer Aldwairi, and Muneer Yassein. Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. *Journal of Computational Science*, 25:152–160, 2018.
- [17] V. Kantharaju, H. Suresh, M. Niranjanaamurthy, S. I. Ansarullah, F. Amin, and A. Alabrah. Machine learning based intrusion detection framework for detecting security attacks in internet of things. *Scientific Reports*, 14:30275, 2024.
- [18] Chuanlong Yin, Yuefei Zhu, Jinlong Fei, and Xinzheng He. A deep learning approach for intrusion detection using recurrent neural networks. *IEEE Access*, 5:21954–21961, 2017.
- [19] Nathan Shone, Tran Ngoc N. Ngoc, Vu Dinh Phai, and Qi Shi. A deep learning approach to network intrusion detection. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(1):41–50, 2018.
- [20] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.
- [21] Gwanggil Kim, Seongjun Lee, and Sehun Kim. A novel hybrid intrusion detection method integrating anomaly detection with misuse detection. *Expert Systems with Applications*, 41(4):1690–1700, 2014.
- [22] Krishna Kumar Ponniah and Bharathi Retnaswamy. A novel deep learning based intrusion detection system for the iot-cloud platform with blockchain and data encryption mechanisms. *Journal of Information Security and Applications*, 2023.
- [23] Zihan Wu, Hong Zhang, Peng Wang, and Zhen Sun. Rtids: A robust transformer-based approach for intrusion detection system. *IEEE Access*, 10:64375–64387, 2022.
- [24] Zhen Long et al. A transformer-based network intrusion detection approach for high-dimensional imbalanced data. *Journal of Cloud Computing*, 2024.
- [25] Ruijie Zhao, Mingwei Zhan, Xianwen Deng, Fangqi Li, et al. A novel self-supervised framework based on masked autoencoder for traffic classification. *IEEE/ACM Transactions on Networking*, 32(3):2012–2025, 2024.

- [26] Longlong Li, Yuliang Lu, Guozheng Yang, and Xuehu Yan. End-to-end network intrusion detection based on contrastive learning. *Sensors*, 24(7):2122, 2024.
- [27] Xiaotian Tan et al. Contrastive learning for network intrusion detection. *ACM International Conference Proceeding Series*, 2025.
- [28] Ibtisam A. Alwhbi et al. Encrypted network traffic analysis and classification using machine learning techniques: A comprehensive survey. *IEEE Access*, 2024.
- [29] Wenqi Dong, Jing Yu, Xinjie Lin, Kewei Chen, Qianwei Meng, Qingjun Yuan, and Yongjuan Wang. Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review. *Neurocomputing*, 617:128444, 2025.
- [30] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference*, pages 633–642, 2022.
- [31] Mei-Hui Zhong, Mingwei Lin, Chao Zhang, et al. A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. *Computers & Security*, 141:103821, 2024.
- [32] Xianwen Deng, Jiayi Zhu, Xiaolong Pei, Lei Zhang, Zheming Ling, and Kui Ren Xue. Flow topology-based graph convolutional network for intrusion detection in label-limited iot networks. *IEEE Transactions on Network and Service Management*, 20(1):684–696, 2023.
- [33] Xinggong Zhang et al. Large language models powered network attack detection. *arXiv preprint arXiv:2503.18487*, 2025.